2024.12.14



KOVO

# 배구리그 여자부 순위예측 프로젝트

2팀 (하이큐)
김찬우, 안정현, 이정현, 오수민, 장동준



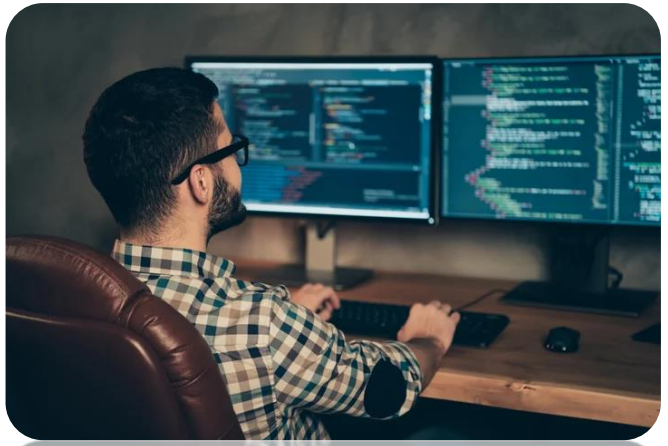
# 역할 분담 및 조원 소개

## 데이터 수집/전처리



김찬우, 안정현, 오수민

## 데이터 분석/모델 설계



김찬우, 안정현, 오수민, 이정현, 장동준



# 목차

1 데이터 수집

2 데이터 전처리

3 데이터 분석과정

4 데이터 분석결과

5 발전 방향

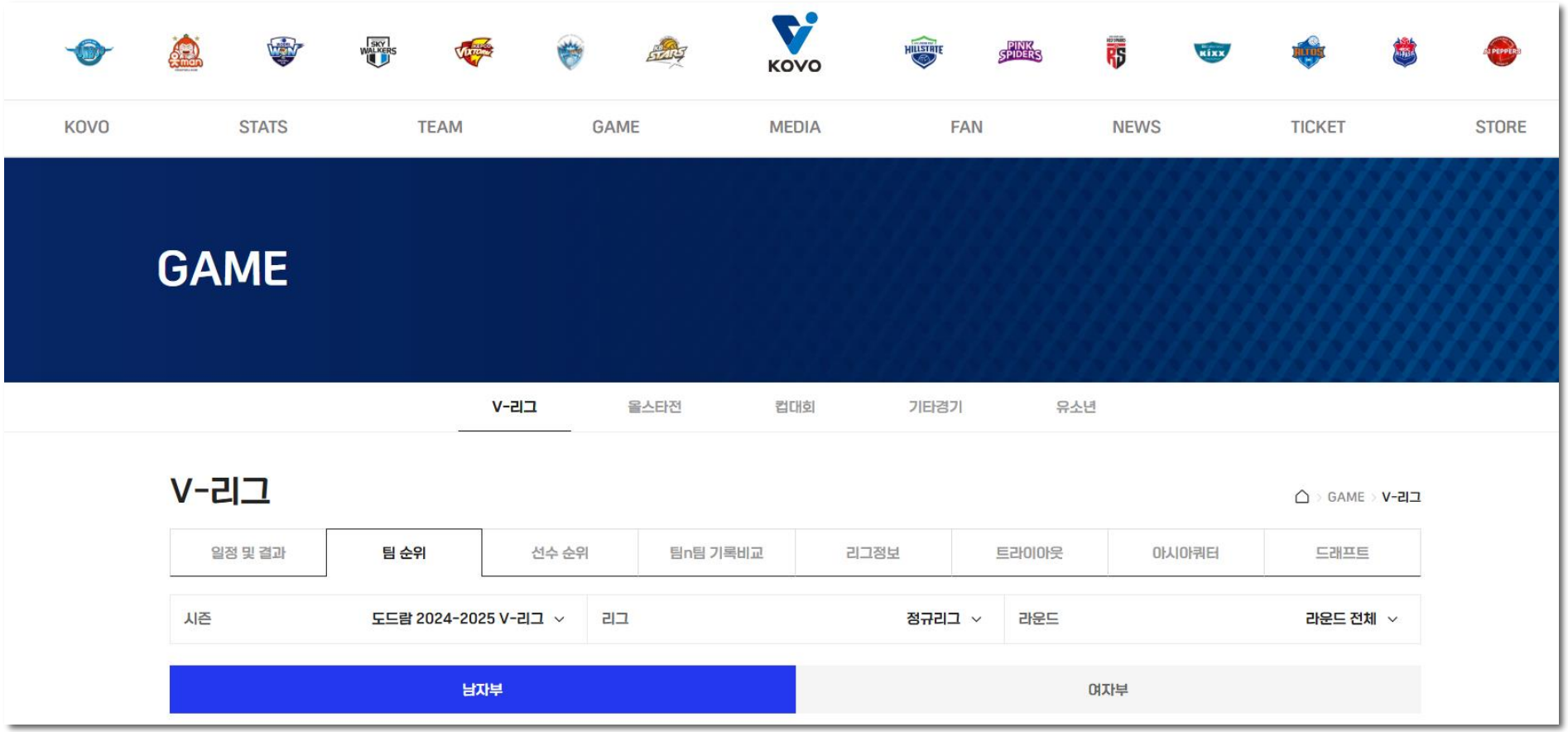
리그 순위 예측 (다중 회귀 모델 기반)

- 팀별 성적 지표(경기당 득실률 등)
- 성적 지표와 리그 최종 승점

# 데이터 수집

KOVO 한국배구연맹 홈페이지 통계 크롤링

https://kovo.co.kr/KOVO/game/v-league?first=%ED%8C%80+%EC%88%9C%EC%9C%84



# 데이터 수집

KOVO 한국배구연맹 홈페이지 통계 크롤링: 첫 화면

2023-2024
2022-2023
2021-2022
2020-2021
2019-2020
2018-2019
2017-2018

일정 및 결과 | 팀 순위 | 선수 순위 | ...그정보 | 트라이아웃 | 아시아쿼터 | 드래프트

시즌 | 도드람 2024-2025 V-리그 | 리그 | 정규리그 | 라운드 | 라운드 전체

남자부 | 여자부

**여자부 선택 이유**

- 여자배구가 남자배구에 비해서 시장규모가 크기 때문에

**팀순위**

| 순위 | 팀 | 승점 | 승 | 패 | 득세트 | 실세트 | 세트득실률 | 득점 | 실점 | 점수득실률 | 연속 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 현대캐피탈 | 29 | 10 | 2 | 32 | 12 | 2.667 | 1071 | 956 | 1.120 | 3승 |
| 2 | 대한항공 | 25 | 8 | 4 | 31 | 19 | 1.632 | 1176 | 1087 | 1.082 | 1패 |
| 3 | 삼성화재 | 18 | 5 | 7 | 22 | 24 | 0.917 | 1048 | 1034 | 1.014 | 1승 |
| 4 | 우리카드 | 17 | 6 | 6 | 24 | 25 | 0.960 | 1111 | 1139 | 0.975 | 2패 |
| 5 | 한국전력 | 16 | 7 | 5 | 23 | 26 | 0.885 | 1086 | 1118 | 0.971 | 2승 |
| 6 | KB손해보험 | 13 | 4 | 8 | 19 | 26 | 0.731 | 1000 | 1053 | 0.950 | 1패 |
| 7 | OK저축은행 | 8 | 2 | 10 | 13 | 32 | 0.406 | 995 | 1100 | 0.905 | 4패 |

# 데이터 수집

KOVO 한국배구연맹 홈페이지 통계 크롤링: 2023-2024 시즌 / 여자부




**팀순위**

| 순위 | 팀 | 승점 | 승 | 패 | 득세트 | 실세트 | 세트득실률 | 득점 | 실점 | 점수득실률 | 연속 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 현대건설 | 80 | 26 | 10 | 90 | 51 | 1.765 | 3173 | 2945 | 1.077 | 1승 |
| 2 | 흥국생명 | 79 | 28 | 8 | 93 | 47 | 1.979 | 3245 | 2981 | 1.089 | 2승 |
| 3 | 정관장 | 61 | 20 | 16 | 74 | 63 | 1.175 | 3083 | 2981 | 1.034 | 2패 |
| 4 | GS칼텍스 | 51 | 18 | 18 | 63 | 69 | 0.913 | 2914 | 2933 | 0.994 | 3패 |
| 5 | IBK기업은행 | 51 | 17 | 19 | 70 | 71 | 0.986 | 3114 | 3073 | 1.013 | 2승 |
| 6 | 한국도로공사 | 39 | 12 | 24 | 58 | 85 | 0.682 | 3034 | 3206 | 0.946 | 1패 |
| 7 | 페퍼저축은행 | 17 | 5 | 31 | 39 | 101 | 0.386 | 2840 | 3284 | 0.865 | 1패 |

# 데이터 수집

KOVO 한국배구연맹 홈페이지 통계 크롤링: 라운드 전체

전 시즌/여자부에 대해서 크롤링
1. 라운드 전체
2. 1 Round ~ 6 Round
3. 공격/서브/블로킹

시즌 | 도드람 2023-2024 V-리그 | 리그 | 정규리그 | 라운드 | 라운드 전체

남자부 | 여자부

**팀순위**

| 순위 | 팀 | 승점 | 승 | 패 | 득세트 | 실세트 | 세트득실률 | 득점 | 실점 | 점수득실률 | 연속 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 현대건설 | 80 | 26 | 10 | 90 | 51 | 1.765 | 3173 | 2945 | 1.077 | 1승 |
| 2 | 흥국생명 | 79 | 28 | 8 | 93 | 47 | 1.979 | 3245 | 2981 | 1.089 | 2승 |
| 3 | 정관장 | 61 | 20 | 16 | 74 | 63 | 1.175 | 3083 | 2981 | 1.034 | 2패 |
| 4 | GS칼텍스 | 51 | 18 | 18 | 63 | 69 | 0.913 | 2914 | 2933 | 0.994 | 3패 |
| 5 | IBK기업은행 | 51 | 17 | 19 | 70 | 71 | 0.986 | 3114 | 3073 | 1.013 | 2승 |
| 6 | 한국도로공사 | 39 | 12 | 24 | 58 | 85 | 0.682 | 3034 | 3206 | 0.946 | 1패 |
| 7 | 페퍼저축은행 | 17 | 5 | 31 | 39 | 101 | 0.386 | 2840 | 3284 | 0.865 | 1패 |

# 데이터 수집

KOVO 한국배구연맹 홈페이지 통계 크롤링: 1 Round ~ 6 Round

전 시즌/여자부에 대해서 크롤링
1. 라운드 전체
2. 1 Round ~ 6 Round
3. 공격/서브/블로킹

일정 및 결과 | 팀 순위 | 선수 순위 | 팀n팀 기록비교 | 리그정보 | 트라이아웃 | 아시아쿼터 | 드래프트

시즌: 도드람 2022-2023 V-리그 | 리그: 정규리그 | 라운드: 6 Round

남자부 | 여자부

**팀순위**

| 순위 | 팀 | 승점 | 승 | 패 | 득세트 | 실세트 | 세트득실률 | 득점 | 실점 | 점수득실률 | 연속 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 흥국생명 | 13 | 4 | 2 | 15 | 8 | 1.875 | 521 | 483 | 1.079 | 3승 |
| 2 | 한국도로공사 | 12 | 4 | 2 | 15 | 9 | 1.667 | 548 | 489 | 1.121 | 4승 |
| 3 | KGC인삼공사 | 10 | 4 | 2 | 12 | 12 | 1.000 | 528 | 513 | 1.029 | 1승 |
| 4 | GS칼텍스 | 9 | 3 | 3 | 11 | 11 | 1.000 | 499 | 477 | 1.046 | 2패 |
| 5 | 현대건설 | 8 | 3 | 3 | 13 | 13 | 1.000 | 554 | 567 | 0.977 | 3패 |
| 6 | IBK기업은행 | 8 | 2 | 4 | 12 | 15 | 0.800 | 564 | 586 | 0.962 | 2패 |
| 7 | 페퍼저축은행 | 3 | 1 | 5 | 7 | 17 | 0.412 | 437 | 536 | 0.815 | 1승 |

# 데이터 수집

KOVO 한국배구연맹 홈페이지 통계 크롤링: 부문별 팀순위 첫 화면

전 시즌/여자부에 대해서 크롤링
1. 라운드 전체
2. 1 Round ~ 6 Round
3. 공격/서브/블로킹

**부문별 팀순위**

득점 | 공격 | 오픈공격 | 시간차공격 | 이동공격 | 후위공격 | 속공 | 퀵오픈 | 서브 | 블로킹 | 디그 | 세트 | 리시브 | 벌칙 | 범실

첫 화면에는 득점에 대한 데이터가 존재

| 순위 | | 경기수 | 세트수 | 공격 | 블로킹 | 서브 | 득점 |
|---|---|---|---|---|---|---|---|
| 1 | 대한항공 | 12 | 50 | 690 | 126 | 71 | 1176 |
| 2 | 우리카드 | 12 | 49 | 667 | 124 | 41 | 1111 |
| 3 | 한국전력 | 12 | 49 | 631 | 123 | 41 | 1086 |
| 4 | 현대캐피탈 | 12 | 44 | 608 | 107 | 71 | 1071 |
| 5 | 삼성화재 | 12 | 46 | 609 | 115 | 35 | 1048 |
| 6 | KB손해보험 | 12 | 45 | 582 | 105 | 55 | 1000 |
| 7 | OK저축은행 | 12 | 45 | 588 | 102 | 26 | 995 |

# 데이터 수집

KOVO 한국배구연맹 홈페이지 통계 크롤링: 공격 → 서브 → 블로킹

전 시즌/여자부에 대해서 크롤링
1. 라운드 전체
2. 1 Round ~ 6 Round
3. 공격/서브/블로킹

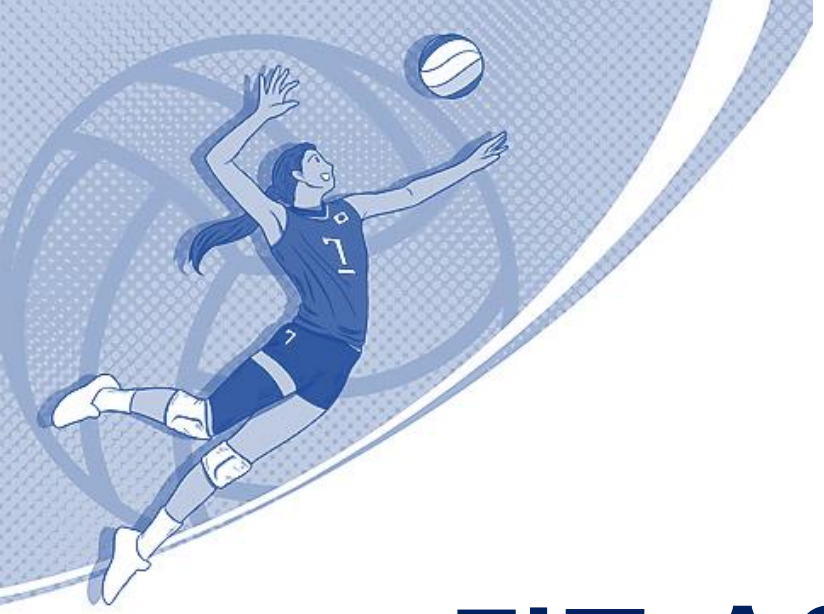


부문별 팀순위 | 득점 | 공격 | 오픈공격 | 시간차공격 | 이동공격 | 후위공격 | 속공 | 퀵오픈 | 서브 | 블로킹 | 디그 | 세트 | 리시브 | 벌칙 | 범실

| 순위 | 팀 | 경기수 | 세트수 | 시도 | 성공 | 공격차단 | 범실 | 성공률 |
|---|---|---|---|---|---|---|---|---|
| 1 | 현대캐피탈 | 12 | 44 | 1121 | 608 | 85 | 70 | 54.24 |
| 2 | 대한항공 | 12 | 50 | 1334 | 690 | 100 | 104 | 51.72 |
| 3 | 한국전력 | 12 | 49 | 1221 | 631 | 120 | 107 | 51.68 |
| 4 | 삼성화재 | 12 | 46 | 1199 | 609 | 114 | 82 | 50.79 |
| 5 | 우리카드 | 12 | 49 | 1323 | 667 | 130 | 107 | 50.42 |
| 6 | KB손해보험 | 12 | 45 | 1177 | 582 | 103 | 74 | 49.45 |
| 7 | OK저축은행 | 12 | 45 | 1266 | 588 | 150 | 78 | 46.45 |

공격/서브/블로킹 선택 이유

- 오픈공격/시간차공격/이동공격/후위공격/속공이 공격에 포함
- 범실, 벌칙의 경우 리그순위와 겹침



# 리그 순위 예측 (다중 회귀 모델 기반)

1 팀별 성적 지표(경기당 득실률 등)




# 데이터 전처리

## 변수 타입 확인

| | | | |
|---|---|---|---|
| 0 | 시즌 | 45 non-null | object |
| 1 | 순위 | 45 non-null | int64 |
| 2 | 팀 | 45 non-null | object |
| 3 | 승점 | 45 non-null | int64 |
| 4 | 승 | 45 non-null | int64 |
| 5 | 패 | 45 non-null | int64 |
| 6 | 득세트 | 45 non-null | int64 |
| 7 | 실세트 | 45 non-null | int64 |
| 8 | 세트득실률 | 45 non-null | float64 |
| 9 | 득점 | 45 non-null | int64 |
| 10 | 실점 | 45 non-null | int64 |
| 11 | 점수득실률 | 45 non-null | float64 |
| 12 | 연속 | 45 non-null | object |
| 13 | 공격_순위 | 45 non-null | int64 |
| 14 | 경기수 | 45 non-null | int64 |
| 15 | 세트수 | 45 non-null | int64 |

- **“시즌”, “팀“, “연속“** 이렇게 세 컬럼의 데이터타입이 **object**이고 나머지는 수치형임을 알 수 있음
- **“팀”**은 **원핫 인코딩, “시즌”**과 **“연속 ”** 컬럼은 **수치형으로 변환**하는 전처리를 진행해야 함




# 데이터 전처리

## 수치형으로 변환 및 원핫 인코딩; 코드

```python
# '연속' 컬럼에서 승/패와 그 횟수를 추출하여 수치로 변환
def convert_streak(streak):
    if '승' in streak:
        return int(streak.replace('승', ''))  # 승리 연승
    elif '패' in streak:
        return -int(streak.replace('패', ''))  # 패배 연패

# 변환된 결과를 새로운 컬럼 '연속_수치'에 추가
table_data['연속_수치'] = table_data['연속'].apply(convert_streak)
table_data = table_data.drop(['연속'], axis = 1)  # 기존 "연속" 컬럼 제거
```

수치형으로 변환

```python
# '시즌' 컬럼에서 시작 연도 추출 및 수치형으로 변환
table_data['시즌'] = table_data['시즌'].str.split('-').str[0].astype(int) + 2000  # 2000을 더하여 연도로 변환
```

```python
# "팀" 컬럼을 원핫 인코딩하고 기존 "팀" 컬럼을 제거
encoded = pd.get_dummies(table_data['팀'], prefix='팀명')
table_data = pd.concat([table_data, encoded], axis=1)
table_data = table_data.drop(['팀'], axis = 1)  # 기존 "팀" 컬럼 제거
```

원핫인코딩

# 데이터 전처리

## 수치형으로 변환; 시즌, 연속

| | | | |
|---|---|---|---|
| 0 | 시즌 | 45 non-null | object |
| 1 | 순위 | 45 non-null | int64 |
| 2 | 팀 | 45 non-null | object |
| 3 | 승점 | 45 non-null | int64 |
| 4 | 승 | 45 non-null | int64 |
| 5 | 패 | 45 non-null | int64 |
| 6 | 득세트 | 45 non-null | int64 |
| 7 | 실세트 | 45 non-null | int64 |
| 8 | 세트득실률 | 45 non-null | float64 |
| 9 | 득점 | 45 non-null | int64 |
| 10 | 실점 | 45 non-null | int64 |
| 11 | 점수득실률 | 45 non-null | float64 |
| 12 | 연속 | 45 non-null | object |
| 13 | 공격_순위 | 45 non-null | int64 |
| 14 | 경기수 | 45 non-null | int64 |
| 15 | 세트수 | 45 non-null | int64 |
| 16 | 공격_시도 | 45 non-null | int64 |
| 17 | 공격_성공 | 45 non-null | int64 |
| 18 | 공격차단 | 45 non-null | int64 |
| 19 | 공격_범실 | 45 non-null | int64 |
| 20 | 공격_성공률 | 45 non-null | float64 |

수치형으로 변환

| | |
|---|---|
| 시즌 | int32 |
| 순위 | int64 |
| 승점 | int64 |
| 승 | int64 |
| 패 | int64 |
| 득세트 | int64 |
| 실세트 | int64 |
| 세트득실률 | float64 |

| | 시즌 | 순위 |
|---|---|---|
| 0 | 23-24 | 1 |
| 1 | 23-24 | 2 |
| 2 | 23-24 | 3 |

| | 시즌 | 순위 |
|---|---|---|
| 0 | 2023 | 1 |
| 1 | 2023 | 2 |
| 2 | 2023 | 3 |

수치형으로 변환

| | |
|---|---|
| 블로킹_순위 | int64 |
| 블로킹_시도 | int64 |
| 블로킹_성공 | int64 |
| 블로킹_유효블락 | int64 |
| 블로킹_실패 | int64 |
| 블로킹_범실 | int64 |
| 블로킹_어시스트 | int64 |
| 블로킹_세트당평균 | float64 |
| 연속_수치 | int64 |

# 데이터 전처리

## 원핫 인코딩; 팀

**원핫인코딩 처리 전**

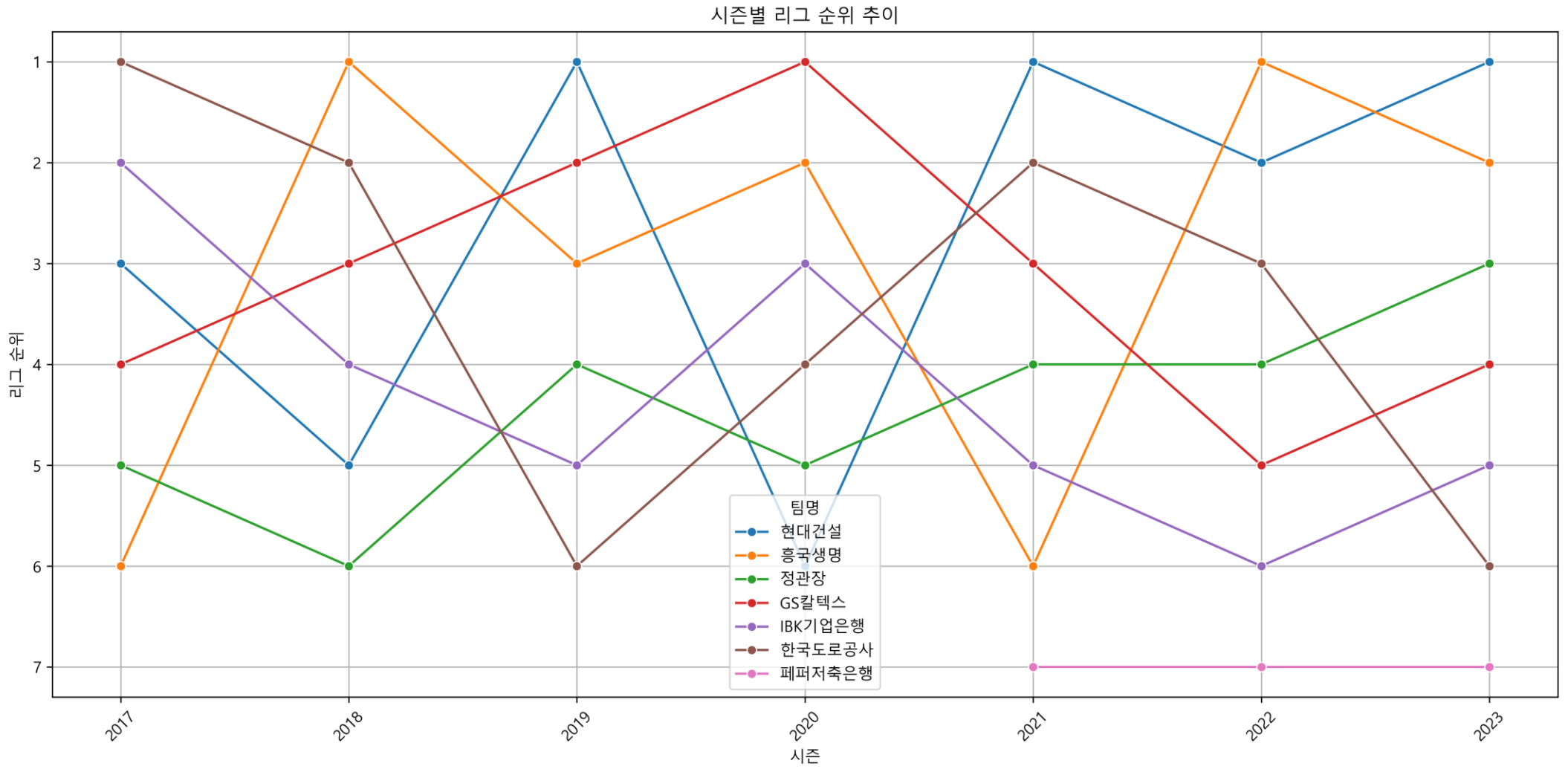
| | 시즌 | 순위 | 팀 | 승점 | 승 | 패 | 득세트 | 실세트 | 세트득실률 | 득점 | ... | 서브_범실 | 서브_세트당평균 | 블로킹_순위 | 블로킹_시도 | 블로킹_성공 | 블로킹_유효블락 | 블로킹_실패 | 블로킹_범실 | 블로킹_어시스트 | 블로킹_세트당평균 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 23-24 | 1 | 현대건설 | 80 | 26 | 10 | 90 | 51 | 1.765 | 3173 | ... | 227 | 0.96 | 1 | 2462 | 337 | 934 | 788 | 28 | 269 | 2.39 |
| **1** | 23-24 | 2 | 흥국생명 | 79 | 28 | 8 | 93 | 47 | 1.979 | 3245 | ... | 243 | 1.19 | 3 | 2422 | 299 | 1014 | 690 | 37 | 219 | 2.14 |
| **2** | 23-24 | 3 | 정관장 | 61 | 20 | 16 | 74 | 63 | 1.175 | 3083 | ... | 284 | 1.00 | 4 | 2212 | 292 | 874 | 687 | 30 | 220 | 2.13 |

**원핫인코딩 처리 후**

| | 시즌 | 순위 | 승점 | 승 | 패 | 득세트 | 실세트 | 세트득실률 | 득점 | 실점 | ... | 블로킹_어시스트 | 블로킹_세트당평균 | 연속_수치 | 팀명_GS칼텍스 | 팀명_IBK기업은행 | 팀명_정관장 | 팀명_페퍼저축은행 | 팀명_한국도로공사 | 팀명_현대건설 | 팀명_흥국생명 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 2023 | 1 | 80 | 26 | 10 | 90 | 51 | 1.765 | 3173 | 2945 | ... | 269 | 2.39 | 1 | False | False | False | False | False | True | False |
| **1** | 2023 | 2 | 79 | 28 | 8 | 93 | 47 | 1.979 | 3245 | 2981 | ... | 219 | 2.14 | 2 | False | False | False | False | False | False | True |
| **2** | 2023 | 3 | 61 | 20 | 16 | 74 | 63 | 1.175 | 3083 | 2981 | ... | 220 | 2.13 | -2 | False | False | True | False | False | False | False |

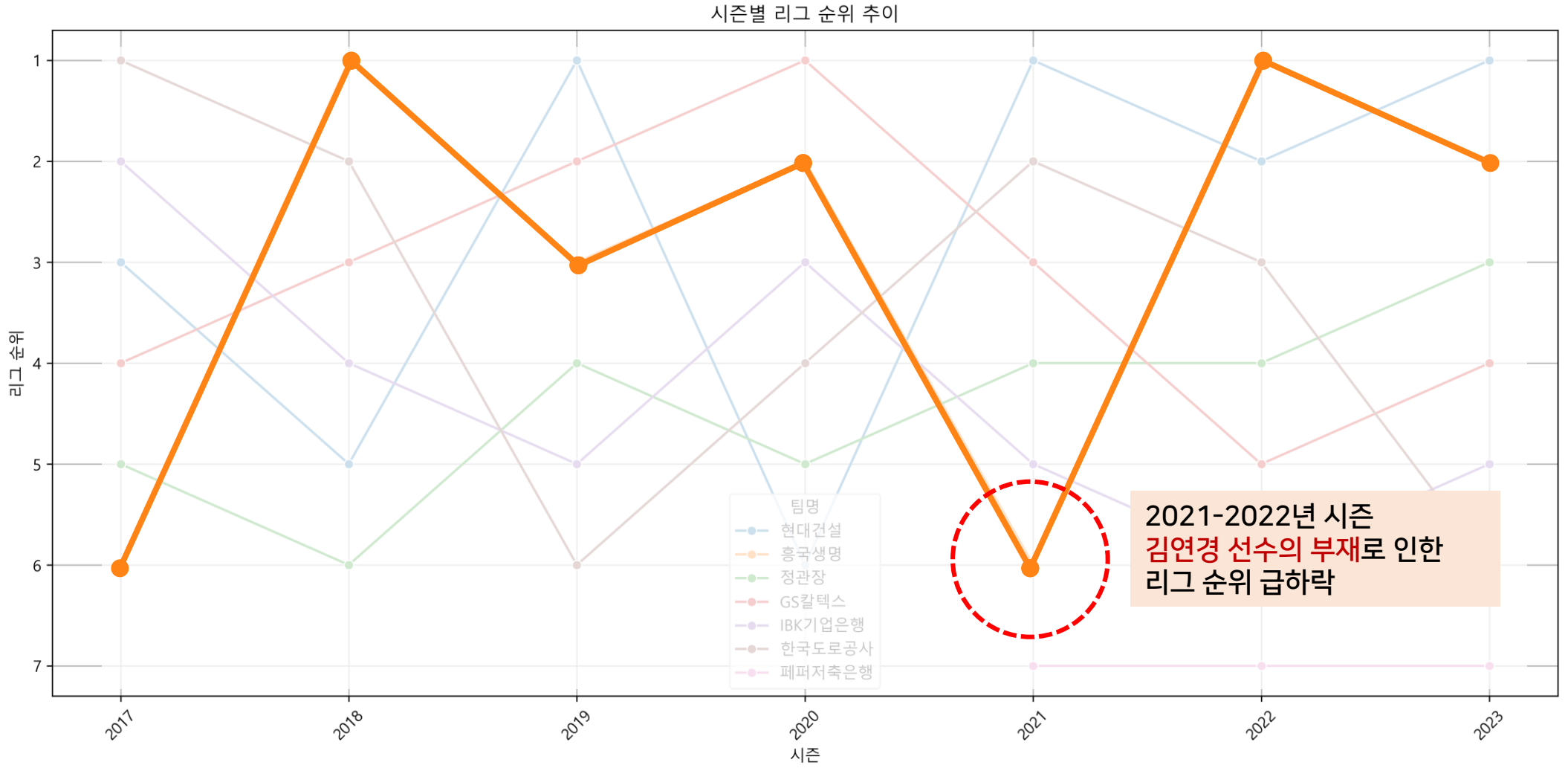
# 데이터 분석과정

## 시즌별 리그 현황



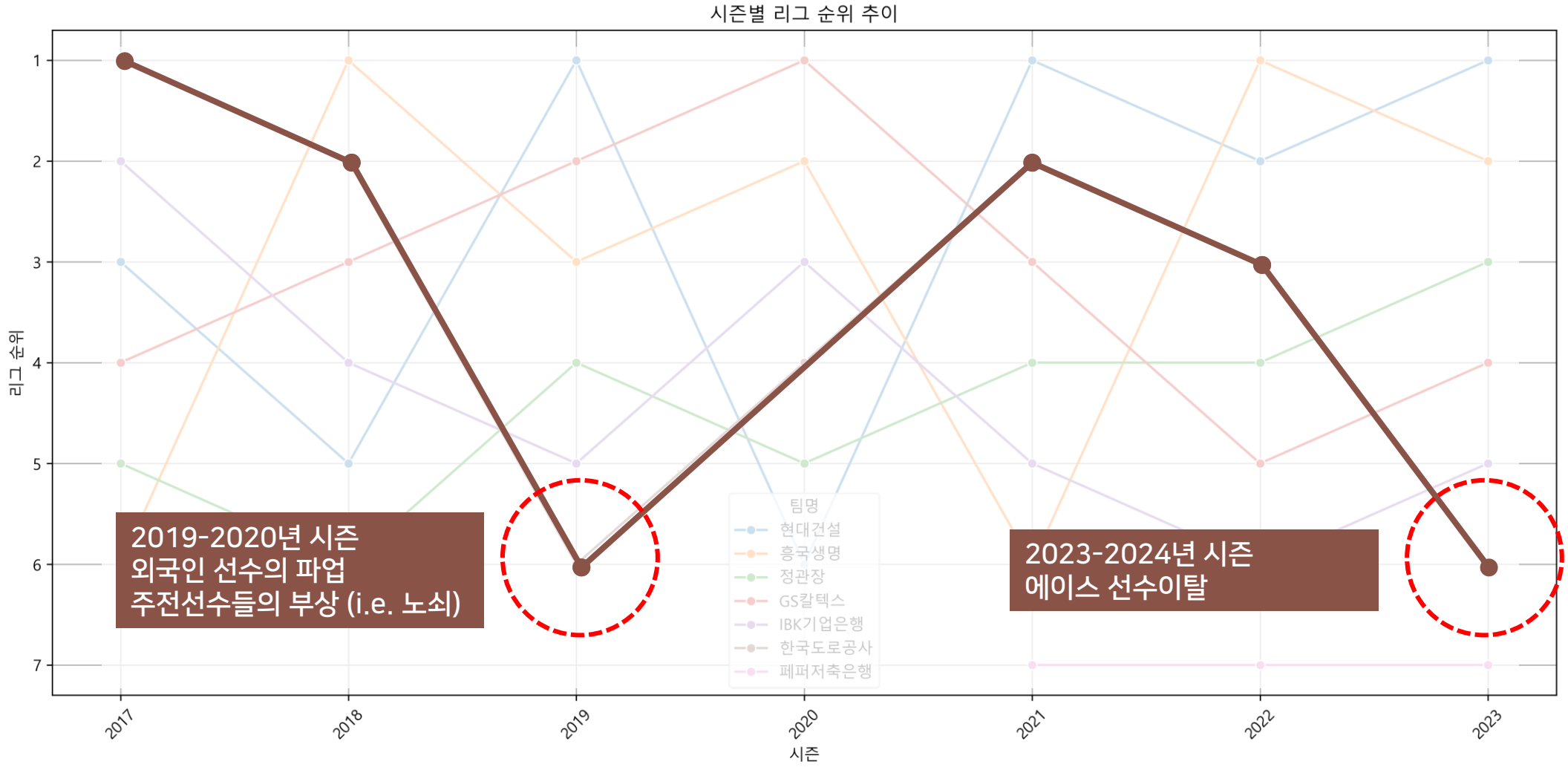
# 데이터 분석과정

## 흥국생명 현황



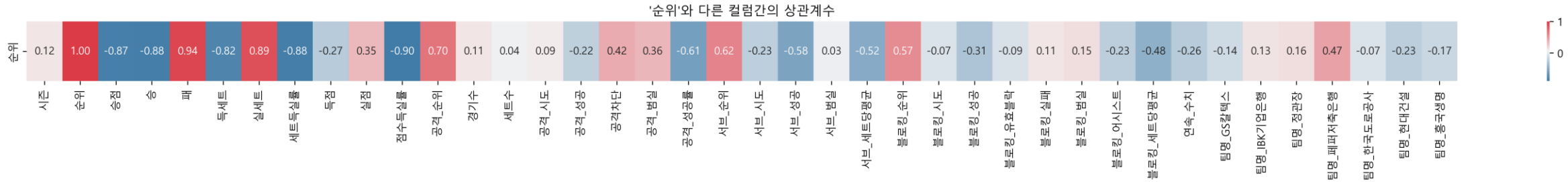
# 데이터 분석과정

## 한국도로공사 현황



# 데이터 분석과정

## 다중상관분석을 통한 변수 선택




상관계수 0.6 이상인 컬럼들 & 0.7이상인 컬럼들로 각각 모델을 만들어 성능을 비교할 예정

### Model 1: 상관계수 0.6 이상

① 승점
② 승
③ 패
④ 득세트
⑤ 실세트
⑥ 세트득실률
⑦ 점수득실률
⑧ 공격_순위
⑨ 공격_성공률
⑩ 서브_순위

### Model 2: 상관계수 0.7 이상

① 승점
② 승
③ 패
④ 득세트
⑤ 실세트
⑥ 세트득실률
⑦ 점수득실률
⑧ 공격_순위




# 데이터 분석과정

## 모델링; 코드

```
# 상관계수 0.6 이상
df1 = table_data[["순위", "승점", "승", "패", "득세트", "실세트", "세트득실률", "점수득실률", "공격_순위", "공격_성공률", "서브_순위"]]
# 상관계수 0.7이상
df2 = table_data[["순위", "승점", "승", "패", "득세트", "실세트", "세트득실률", "점수득실률", "공격_순위"]]
```

```
x1 = df1.drop("순위", axis = 1)  # 독립 변수
y1 = df1["순위"]  # 종속 변수

x2 = df2.drop("순위", axis = 1)  # 독립 변수
y2 = df2["순위"]  # 종속 변수
```

```
# 데이터셋 분할
x1_train, x1_test, y1_train, y1_test = train_test_split(x1, y1, test_size = 0.2, stratify = y1, random_state = 12)
x2_train, x2_test, y2_train, y2_test = train_test_split(x2, y2, test_size = 0.2, stratify = y2, random_state = 12)
```

```
model1 = LinearRegression()
model1.fit(x1_train, y1_train)
```

```
model2 = LinearRegression()
model2.fit(x2_train, y2_train)
```

# 데이터 분석 결과

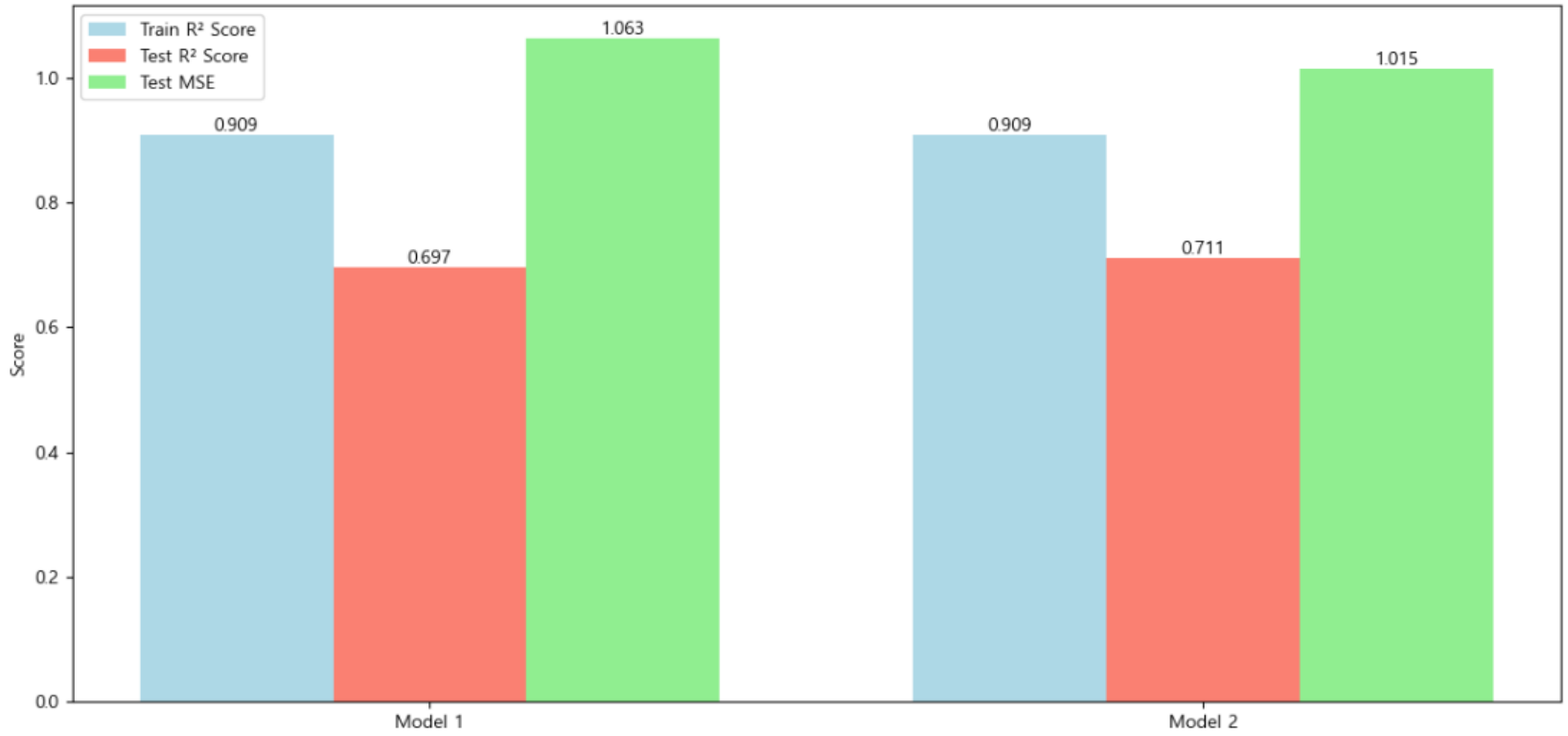
## Model 1 vs Model 2 성능비교: 결정계수 및 MSE

- Model 1: 상관계수 0.6 이상인 컬럼을 독립변수로 훈련
- Model 2: 상관계수 0.7이상인 컬럼을 독립변수로 훈련

Model 1, Model 2의 성능비교

```
print(model1.score(x1_train, y1_train), model1.score(x1_test, y1_test))
print(model2.score(x2_train, y2_train), model2.score(x2_test, y2_test))
```

```
0.9089179116726159 0.6969030057207348
0.9085269406042681 0.7105127300764571
```




결정계수 및 MSE로 Model 1과 Model 2 성능비교 시각화

# 데이터 분석 결과

## 결과 1. 리그 순위 예측

### 2024-2025 현재 시즌

| 순위 | 팀 |
|---|---|
| 1 | 흥국생명 |
| 2 | 현대건설 |
| 3 | 정관장 |
| 4 | IBK기업은행 |
| 5 | 페퍼저축은행 |
| 6 | 한국도로공사 |
| 7 | GS칼텍스 |

### 2024-2025 시즌 **최종순위 예측**

| 순위 | 팀 |
|---|---|
| 1 | 현대건설 |
| 2 | 흥국생명 |
| 3 | IBK기업은행 |
| 4 | 정관장 |
| 5 | 페퍼저축은행 |
| 6 | 한국도로공사 |
| 7 | GS칼텍스 |

현대건설 1위로 변경
흥국생명 2위로 변경

# 데이터 분석 결과

## 왜 2024-2025 시즌 최종순위 1위가 현대건설인가?

2024-2025 현재 시즌

| 순위 | 팀 |
|---|---|
| 1 | 흥국생명 |
| 2 | 현대건설 |

2024-2025 시즌 **최종순위 예측**

| 순위 | 팀 |
|---|---|
| 1 | 현대건설 |
| 2 | 흥국생명 |

**팀순위**

| 순위 | 팀 | 승점 | 승 | 패 | 득세트 | 실세트 | 세트득실률 | 득점 | 실점 | 점수득실률 | 연속 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 흥국생명 | 40 | 14 | 0 | 42 | 10 | 4.200 | 1265 | 1038 | 1.219 | 14승 |
| 2 | 현대건설 | 31 | 10 | 4 | 36 | 19 | 1.895 | 1292 | 1140 | 1.133 | 1패 |

흥국생명이 최종순위가 1등일 것이라고 예측함

# 데이터 분석 결과

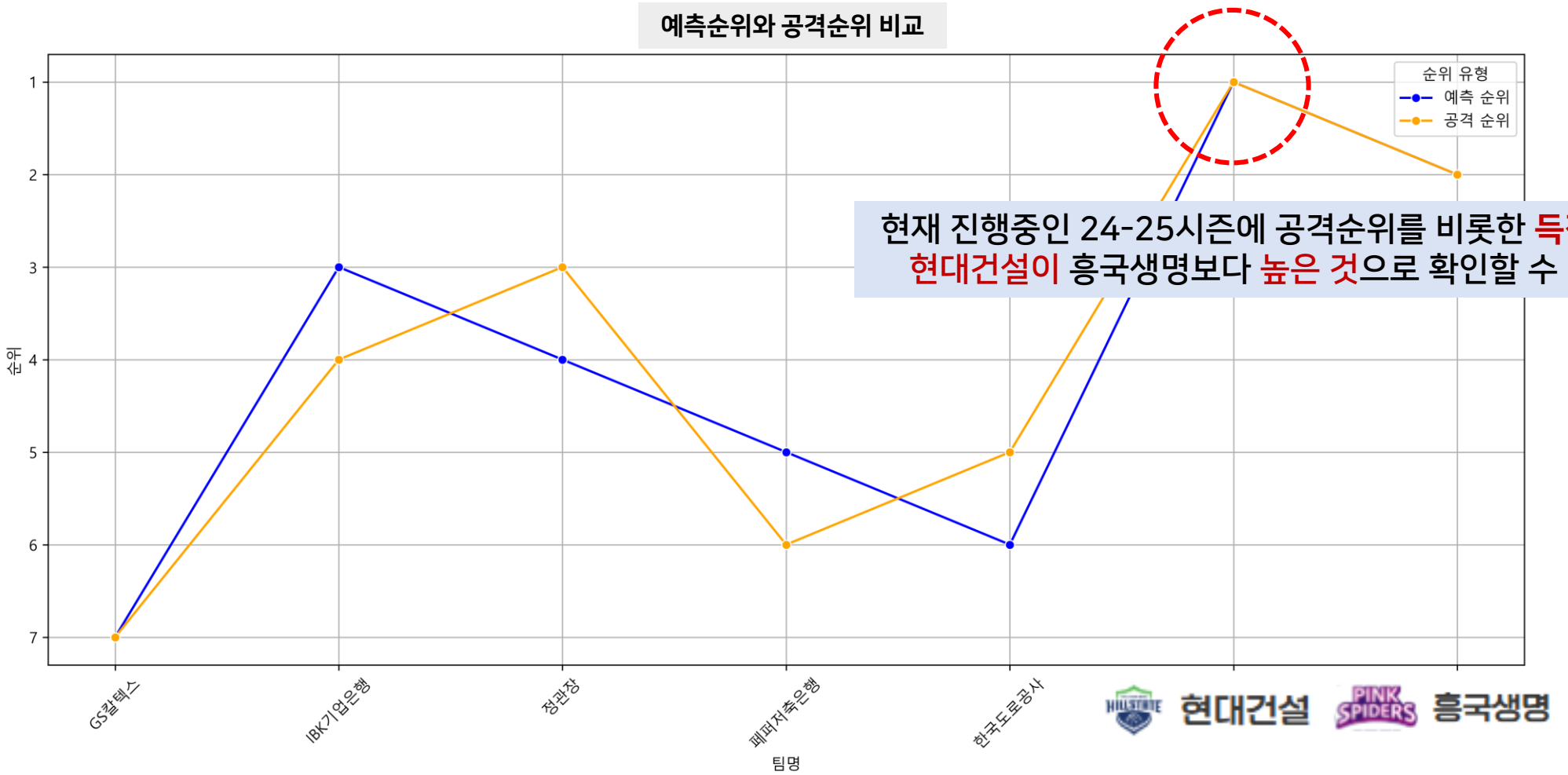
## 왜 2024-2025 시즌 최종순위 1위가 현대건설인가?

**2024-2025 현재 시즌**

| 순위 | 팀 |
|---|---|
| 1 | 흥국생명 |
| 2 | 현대건설 |

**2024-2025 시즌 최종순위 예측**

| 순위 | 팀 |
|---|---|
| 1 | 현대건설 |
| 2 | 흥국생명 |




현재 진행중인 24-25시즌에 공격순위를 비롯한 **득점지표**가
**현대건설**이 흥국생명보다 **높은 것**으로 확인할 수 있었음

# 데이터 분석 결과

## 결과 1. 리그 순위 예측

### 2024-2025 현재 시즌

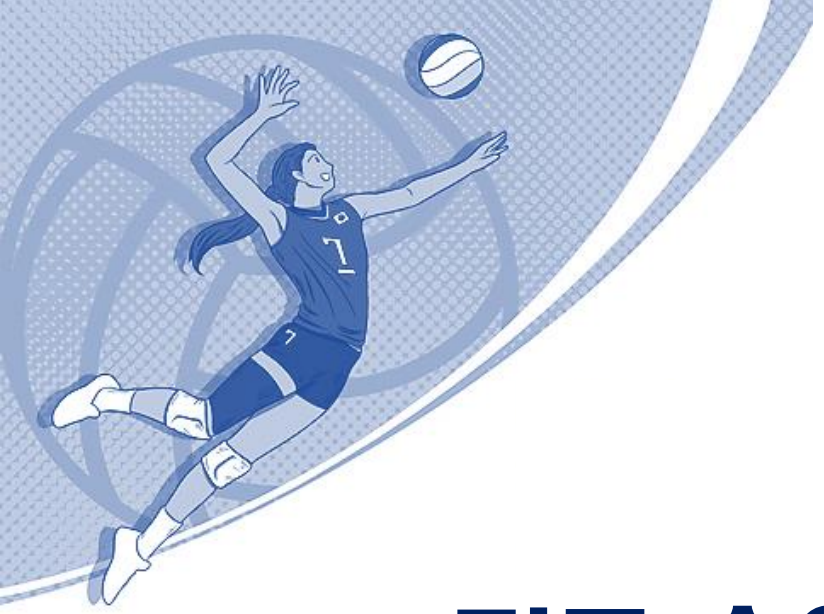
| 순위 | 팀 |
|---|---|
| 1 | 흥국생명 |
| 2 | 현대건설 |
| 3 | 정관장 |
| 4 | IBK기업은행 |
| 5 | 페퍼저축은행 |
| 6 | 한국도로공사 |
| 7 | GS칼텍스 |

### 2024-2025 시즌 **최종순위 예측**

| 순위 | 팀 |
|---|---|
| 1 | 현대건설 |
| 2 | 흥국생명 |
| 3 | IBK기업은행 |
| 4 | 정관장 |
| 5 | 페퍼저축은행 |
| 6 | 한국도로공사 |
| 7 | GS칼텍스 |

현대건설 1위로 변경
흥국생명 2위로 변경

정관장 4위로 변경
IBK기업은행 3위로 변경



# 리그 순위 예측 (다중 회귀 모델 기반)

## 2 성적 지표와 리그 최종 승점



# 기획의도

## 성적지표와 리그 총점 승점

① 리그 승점 부여방식중 상당히 특이한 규칙 때문 (속칭 "5꽉룰")

② 승패 예측과, 경기지표가 순위로 직관적으로 이어지지 않음, 오히려 승률이 높아도 5꽉룰 때문에 순위가 낮은 경우도 있음.

③ 이러한 특성이 경기지표와 순위간의 비선형성 문제를 불러올 가능성이 크다 판단

④ 따라서 순위가 아닌 "경기당 기대 승점"을 예측하는 방향성으로 모델 제작

# 데이터 분석과정

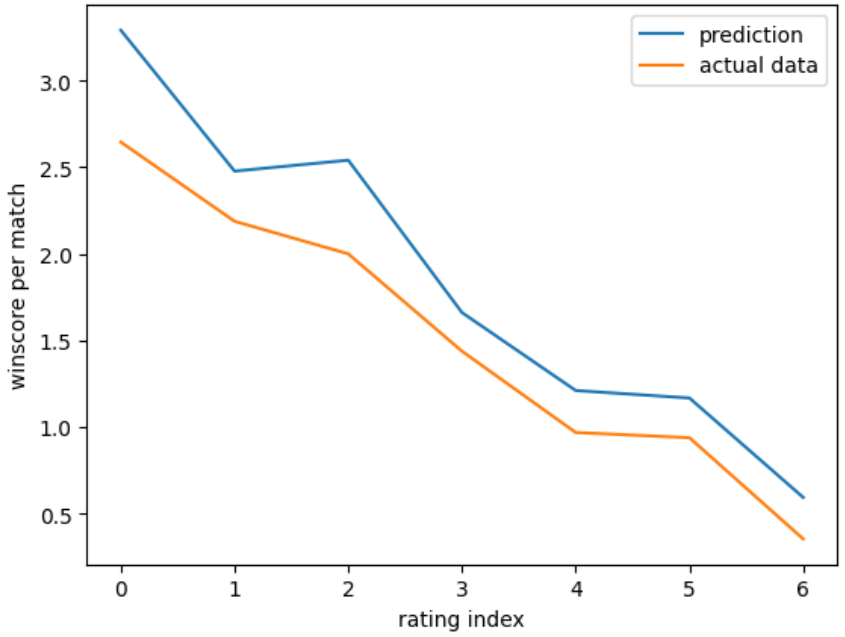
왜 21-22시즌 이후 데이터만 사용했는가?

① 리그 참가 팀 숫자의 차이에서 생기는 데이터 특성 변화

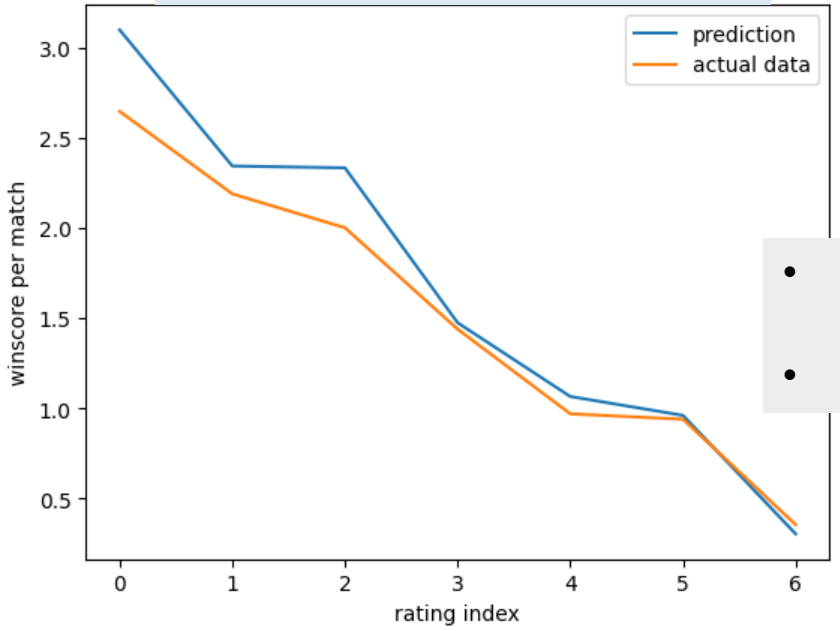
② 20-21시즌 (기존 6팀) → 21-22시즌 7팀으로 변경 ( 페퍼저축은행 리그 참가)

- 흥국생명
- 현대건설
- 정관장
- IBK기업은행
- 페퍼저축은행
- 한국도로공사
- GS칼텍스

**기존 6팀으로 데이터 학습시킨 경우**




**7팀일 때 데이터 학습시킨 경우**




- 선형회귀 모델을 비교해본 결과, 오차율에서 큰 차이를 보임
- 따라서 오차가 적은 7팀으로 채택

# 데이터 분석과정

## 전처리 파트에서 왜 해당 수치들을 사용했는가?

```
pre_x = data_for_p.loc[:,['세트득실률', '점수득실률','공격_성공률','서브_세트당평균', '블로킹_세트당평균']]
```

### 독립변수

| | 세트득실률 | 점수득실률 | 공격_성공률 | 서브_세트당평균 | 블로킹_세트당평균 |
|---|---|---|---|---|---|
| 0 | 0.378 | 0.846 | 34.83 | 1.18 | 2.14 |
| 1 | 1.174 | 1.020 | 37.27 | 1.22 | 1.76 |
| 2 | 1.240 | 1.012 | 39.74 | 1.38 | 2.41 |
| 3 | 0.548 | 0.923 | 36.15 | 0.81 | 2.08 |
| 4 | 0.486 | 0.905 | 36.66 | 1.22 | 1.76 |
| 5 | 1.895 | 1.133 | 41.91 | 1.00 | 2.51 |
| 6 | 3.900 | 1.207 | 41.44 | 1.37 | 2.71 |

**각 팀의 세트 득실률, 점수 득실률, 공격 성공률, 세트당 평균 서브득점, 세트당 평균 블로킹 득점을 포함.**

경기 수에 따라 정비례 하는 수치들을 사용하면 현재 시즌 예측에 사용할 수 없기 때문에, 경기당, 세트당 평균 수치들인 [세트득실률], [점수득실률], [공격_성공률], [서브_세트당평균], [블로킹_세트당평균] 수치들을 선별함

### 종속변수

```
train_y = winscore_p_2324

train_y.extend(winscore_p_2223)
```

종속변수를 기대승점으로 설정

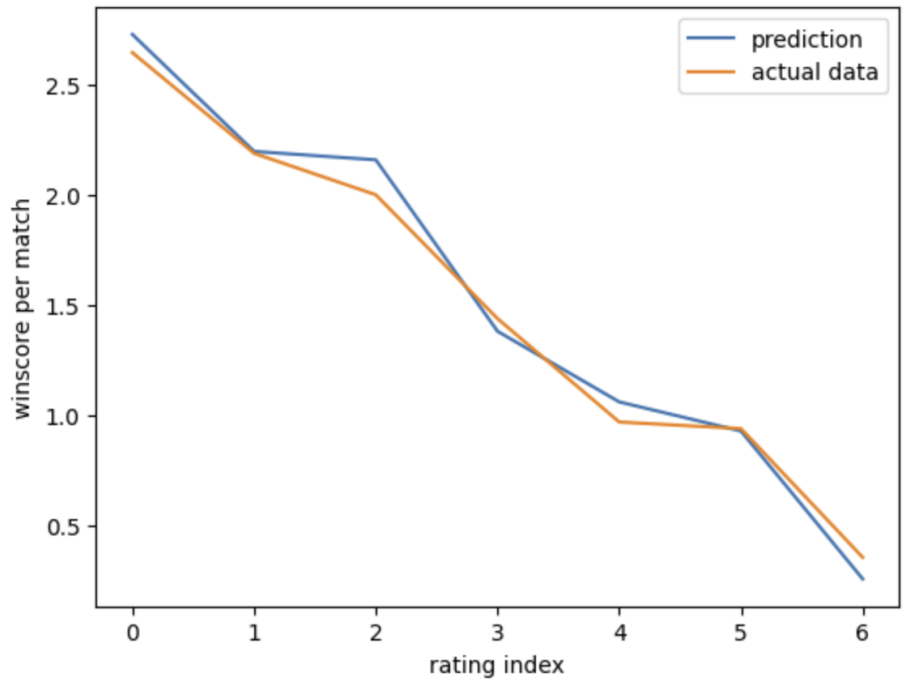
- 기대승점 = 실제 팀 승점 / 경기 수

# 데이터 분석 결과

## 결과 2. 리그순위예측

```
plt.plot(lr.predict(test_x), label = "prediction")
plt.plot(test_y, label = "actual data")
plt.legend()
plt.ylabel("winscore per match")
plt.xlabel("rating index")

plt.show()
```




```
lr = LinearRegression()
```

```
lr.fit(train_x, train_y)
```

LinearRegression

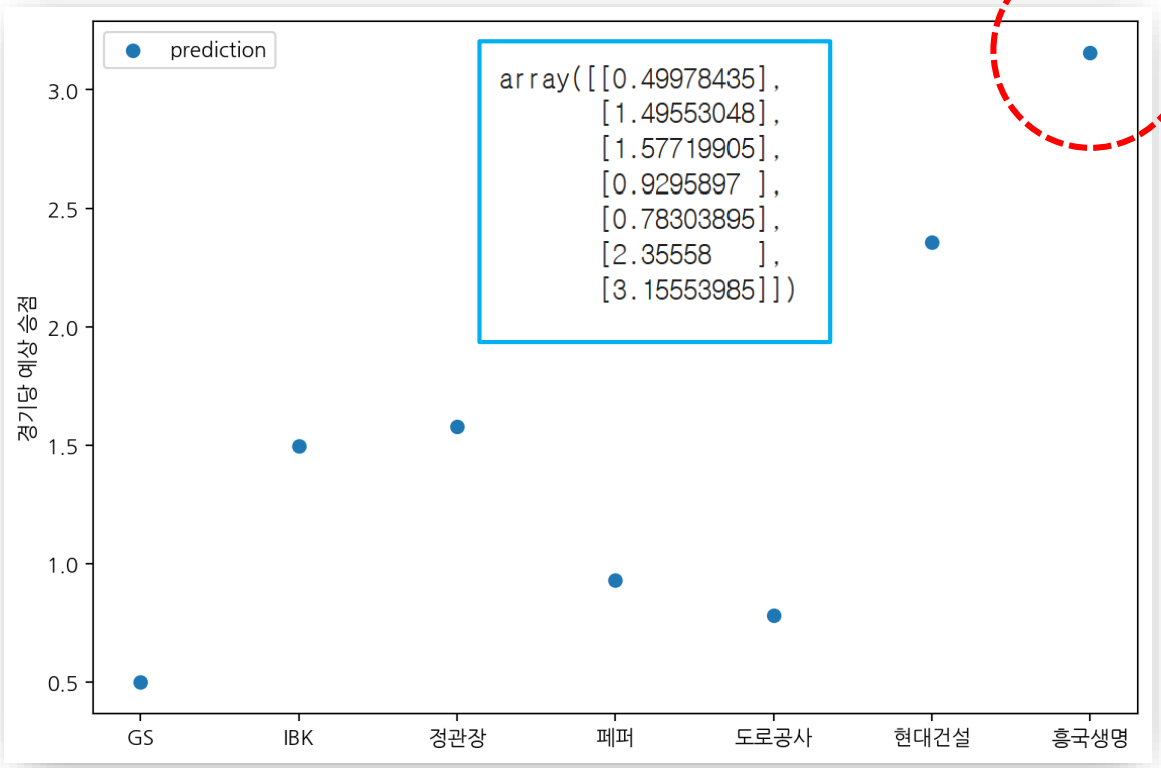
LinearRegression()

흥국생명
현대건설
정관장
IBK기업은행
페퍼저축은행
한국도로공사
GS칼텍스

LinearRegression(선형 회귀)으로 승점을 predict한 결과가 실제 데이터와의 오차가 크지 않음.

# 데이터 분석 결과

## 결과 2. 리그순위예측




1위: 흥국생명

경기당 예상 승점을 통해 순위 예측

- 총 예측 승점 = 경기 수 x 경기당 예상승점

| 순위 | 팀 | |
|---|---|---|
| 1 | 흥국생명 | 108점 |
| 2 | 현대건설 | 85점 |
| 3 | 정관장 | 57점 |
| 4 | IBK기업은행 | 54점 |
| 5 | 페퍼저축은행 | 33점 |
| 6 | 한국도로공사 | 28점 |
| 7 | GS칼텍스 | 18점 |

흥국생명이 독보적 1위로 예측됨
(현 시즌 14연승 중으로 인해 독보적인 세트득실률이 포함되어서 이런 결과가 나온 것으로 보고 있다)

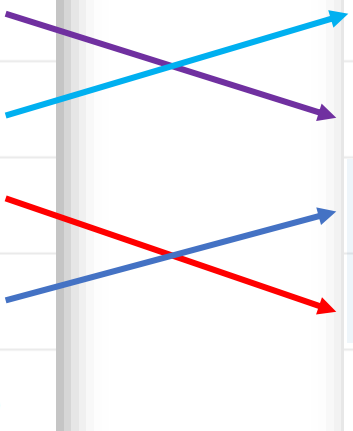


# 결과 최종 정리

# 결과1. 리그순위예측

2024-2025 현재 시즌

| 순위 | 팀 |
|---|---|
| 1 | 흥국생명 |
| 2 | 현대건설 |
| 3 | 정관장 |
| 4 | IBK기업은행 |
| 5 | 페퍼저축은행 |
| 6 | 한국도로공사 |
| 7 | GS칼텍스 |



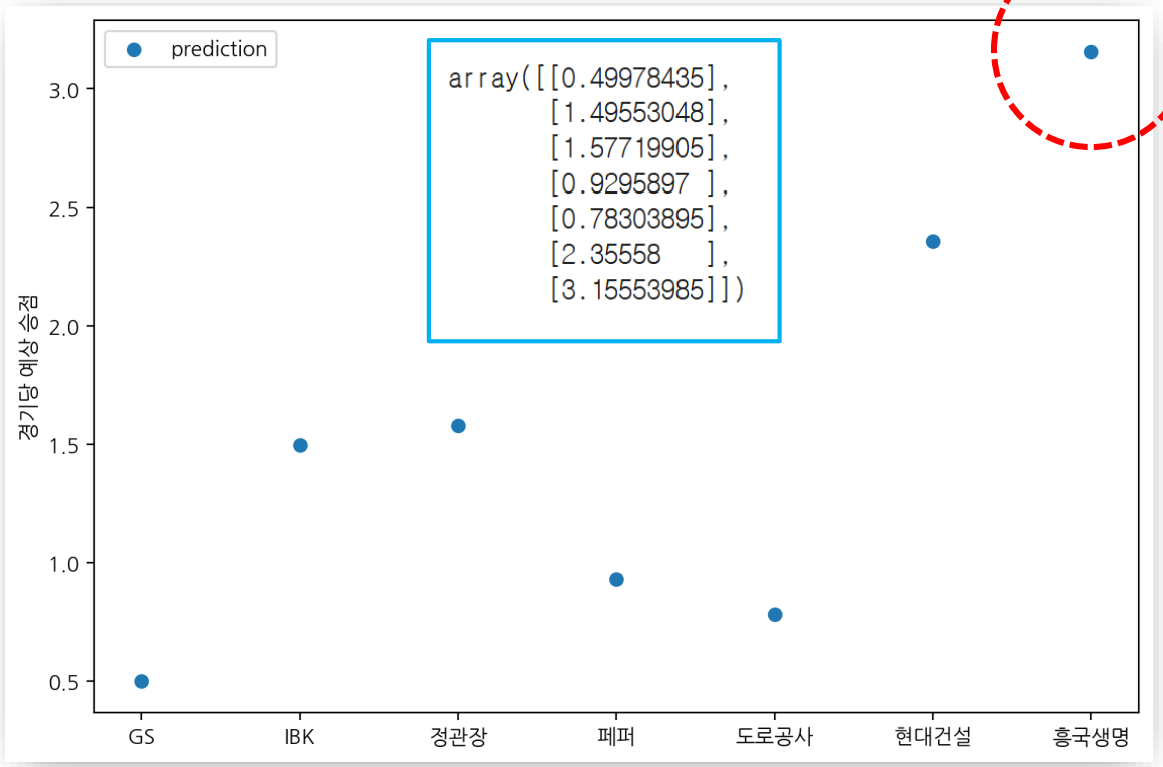
2024-2025 시즌 **최종순위 예측**

| 순위 | 팀 |
|---|---|
| 1 | 현대건설 |
| 2 | 흥국생명 |
| 3 | IBK기업은행 |
| 4 | 정관장 |
| 5 | 페퍼저축은행 |
| 6 | 한국도로공사 |
| 7 | GS칼텍스 |

현대건설 1위로 변경
흥국생명 2위로 변경

정관장 4위로 변경
IBK기업은행 3위로 변경

# 결과2. 리그순위예측




1위 흥국생명

| 순위 | 팀 | |
|---|---|---|
| 1 | 흥국생명 | 108점 |
| 2 | 현대건설 | 85점 |
| 3 | 정관장 | 57점 |
| 4 | IBK기업은행 | 54점 |
| 5 | 페퍼저축은행 | 33점 |
| 6 | 한국도로공사 | 28점 |
| 7 | GS칼텍스 | 18점 |

# 5 발전방향

**라운드 데이터 결합 관련 사항**

- 시즌 23-24에는 “6 round” 데이터 밖에 존재하지 않음
- 나머지 6개 시즌 라운드데이터와 23-24 라운드 데이터를 합침

→ 추후에는 한꺼번에 데이터를 합칠 수 있는 방향을 고려

2023-2024
2022-2023
2021-2022
2020-2021
2019-2020
2018-2019
2017-2018

# 5 발전방향

## KNN 분류모델

[30]:
```
ss = StandardScaler()
scaled_train = ss.fit_transform(x2_train)
scaled_test = ss.transform(x2_test)
```

[31]:
```
knn = KNeighborsClassifier(n_neighbors = 10)
knn.fit(scaled_train, y2_train)
```

[31]:
```
KNeighborsClassifier(n_neighbors=10)
```

```
# test data 예측
y_pred = knn.predict(scaled_test)
pd.DataFrame({"ans" : y2_test, "pred" : y_pred})
```

| | ans | pred |
|---|---|---|
| **38** | 6 | 5 |
| **2** | 3 | 4 |
| **6** | 7 | 5 |
| **14** | 1 | 1 |
| **8** | 2 | 2 |
| **4** | 5 | 3 |
| **24** | 4 | 4 |
| **10** | 4 | 4 |
| **26** | 6 | 5 |

```
knn.score(scaled_test, y2_test)
```

```
0.4444444444444444
```

| | 팀명 | 현재 순위 | 예측 순위 |
|---|---|---|---|
| **0** | GS칼텍스 | 7 | 5 |
| **1** | IBK기업은행 | 4 | 4 |
| **2** | 정관장 | 3 | 5 |
| **3** | 페퍼저축은행 | 5 | 1 |
| **4** | 한국도로공사 | 6 | 2 |
| **5** | 현대건설 | 2 | 3 |
| **6** | 흥국생명 | 1 | 4 |




# 발전방향

## 로지스틱회귀

```
# 로지스틱 회귀 모델 학습
model_logi = LogisticRegression(multi_class='multinomial', solver='lbfgs', max_iter = 100000)
model_logi.fit(x2_train, y2_train)
```

LogisticRegression

```
LogisticRegression(max_iter=100000, multi_class='multinomial')
```

```
y_pred = model_logi.predict(x2_test)
```

```
# 성능 평가
accuracy = accuracy_score(y2_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
```

```
Accuracy: 0.56
```

2024.12.14



# 감사합니다

2팀 (하이큐)
김찬우, 안정현, 이정현, 오수민, 장동준